

自定义函数

无返回值函数:

```
void 函数名(参数)
{
    ...;
    return ;
}
```

有返回值函数:

```
数据类型 函数名(参数)
{
    ...;
    return 返回值;
}
```

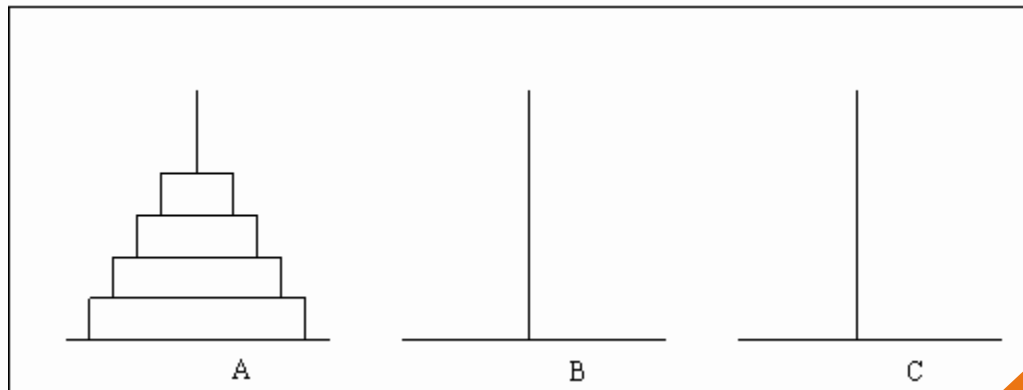
参数可以一个，可以多个也可以一个都没有

无返回值函数只需要一个return就可以了，没有返回值。有返回值函数，返回值的数据类型和函数名前面的数据类型必须一致

有返回值函数，返回之后会把返回值传递给调用他的函数

汉诺塔

法国数学家爱德华·卢卡斯曾编写过一个印度的古老传说：在世界中心贝拿勒斯（在印度北部）的圣庙里，一块黄铜板上插着三根宝石针。印度教的主神梵天在创造世界的时候，在其中一根针上从下到上地穿好了由大到小的64片金片，这就是所谓的汉诺塔。不论白天黑夜，总有一个僧侣在按照下面的法则移动这些金片：一次只移动一片，不管在哪根针上，小片必须在大片上面。僧侣们预言，当所有的金片都从梵天穿好的那根针上移到另外一根针上时，世界就将在一声霹雳中消灭，而梵塔、庙宇和众生也都将同归于尽。



汉诺塔

我们先来玩一下汉诺塔这个游戏，看下哪位同学用的时间最短，步数最少，希望同学们认真思考之后再操作。

把一个n层的汉诺塔从一根柱子移动到另一根柱子，有没有一个最短的步数呢？如果有？是多少呢？有没有办法可以求出来

当n=1时

$m=1;$

当n=2时

$m=3$

当n=3时

$m=7$

当n=4时

$m=15$

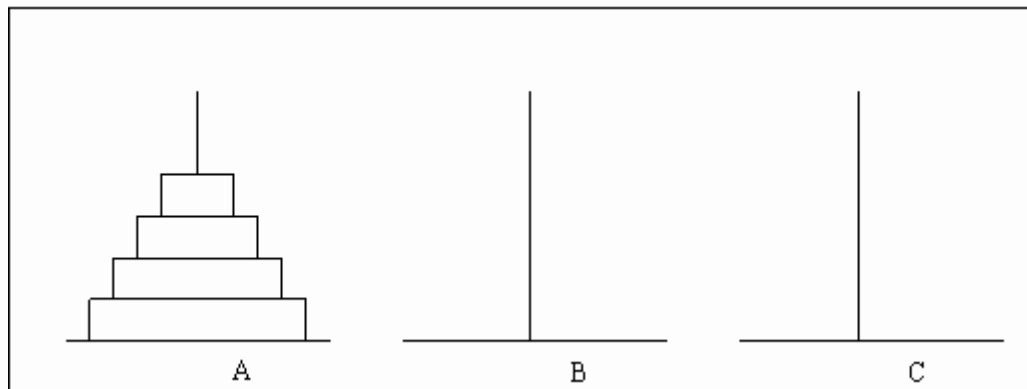
我们可以通过观察得出：

$$f[n]=2^n-1$$

但是有多少同学用了最短步数就成功了呢？

我们可不可以设计出一种程序来求出这种方案数呢？

汉诺塔



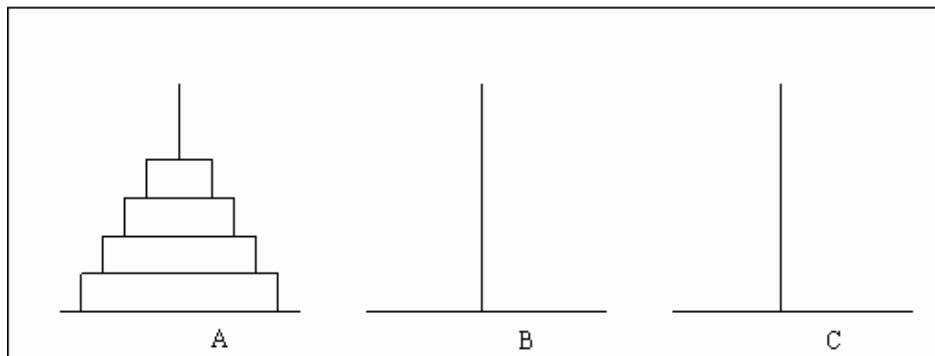
问题分析：

如果我要把柱子A上的 n 个环借助柱子B移动到C上，那么我需要先把A上的 $(n-1)$ 个环借助柱子C移动到B上，再把剩下的一个环直接从A移动到C上，这样最下面那个环就移动好了。然后再把柱子B上面的 $(n-1)$ 个环通过柱子A移动到柱子C上面。这样我们就完成了。

至于我们怎么移动上面 $(n-1)$ 个环，是不是直接重复前面的操作就可以了，把 $(n-1)$ 个环分成上面 $(n-2)$ 个环和最下面一个环。当分到不能自分的时候就结束了。

是不是很神奇！！！！

汉诺塔



每次移动的时候都把当前状态下的上面 $(n-1)$ 个环看成一个整体，下面一个环看成一个整体。

```
void mov(int n,char a,char b,char c)
{
    if(n==0)
        return ;
    mov(n-1,a,c,b); //把前(n-1)个环从柱子a通过柱子c移动到柱子b
    printf("%d %c->%c\n",n,a,c); //把第n个环从a移动到c。
    mov(n-1,b,a,c); //再把前(n-1)个环从柱子b通过柱子a移动到柱子c
    //这样操作之后就移动好了。
}

mov(n,'a','b','c'); //把n个环从柱子a通过柱子b移动到柱子c
```

回溯法

回溯法是递归这种结构的一种常见应用。他的基本思想是：为了求得问题的解，先选择某一种可能的情况向前搜索。在搜索的过程中，一旦发现原来的选择是错误的，或者找到一种正解，就退回上一步重新选择，再继续向前探索。如此反复进行，知道得到所有的解或者证明无解结束。

以前我们在学习递归的时候，使用了一种让递归次数变少的一个方法——记忆化搜索。即用数组记录当前这种状态下的方案种数，后面需要再次求解时就可以直接使用了。同样的，有的时候，回溯法也需要使用到这种方法。这就需要我们观察当前这种状态时候在其他时候也会使用到了。

回溯法框架

```
void dfs(当前状态)
{
    if(到达目标状态)
    {
        记录(输出)结果;
        return;
    }
    for(可取范围)
    {
        if(满足条件)
        {
            标记(记录);
            dfs(下一状态);
            回溯, 取消标记, 寻找其他可能。
        }
    }
    return ;
}
```

字符全排列

给定一个由不同的小写字母组成的字符串，输出这个字符串的所有全排列。我们假设对于小写字母有'a' < 'b' < ... < 'y' < 'z'，而且给定的字符串中的字母已经按照从小到大的顺序排列。字符长度小于等于6。字符串中所有的字符均不相同

输入：

abc

输出：

abc

acb

bac

bca

cab

cba

字符全排列

问题分析：

我们可以把题目理解为有一个长度为 l 的字符串 s 。想要把这些已知的字符放入 $1--l$ 这 l 个盒子中，问有多少种方法。其实就和我们前面的 $1--n$ 的数字的全排列比较类似。只是这里换成了 $s[0], s[1], \dots s[l-1]$ 。

首先，我们还是从第一个盒子开始放，**需要满足条件**

- (1) 该字符是存在于原字符串中的。
- (2) 该字符还没有被放进其他盒子

如果可以放该字符，放入之后，**我们应该做？**

- (1) 标记该字符为已经放过了。
- (2) 用一个数组来存储该位置放的字符是什么
- (3) 走到下一个盒子继续循环执行上一步操作。

字符全排列

```
void dfs(int step)
{
```

字符串下标是s[0]--s[l-1]

```
    if (step==l)      输出:
        for (int i=0;i<l;i++)
```

注意这里不是book[i]，i表示第i个位置，我们应该标记的是第i个位置的字符

```
        if (book[a[i]]==0) //如果该字符没放置过
        {
            book[a[i]]=1; //标记
            b[step]=a[i]; //当前位置放置字符a[i]
            dfs(step+1); //考虑下一个
            book[a[i]]=0; //取消标记，回溯
        }
    }
    return ;
}
```

book数组一定要开大一点，字母对应的ASCLL比较大

```
dfs(0); //字符串是从下标0开始的
```

排列问题

排列与组合是常用的数学方法，其中组合就是从 n 个元素中抽出 r 个元素(不分顺序且 $r \leq n$)，我们可以简单地将 n 个元素理解为自然数 $1, 2, \dots, n$ ，从中任取 r 个数。

输入：

5 3

输出：

1	2	3
1	2	4
1	2	5
1	3	4
1	3	5
1	4	5
2	3	4
2	3	5
2	4	5
3	4	5

仔细观察该题和前面我们
做的题有什么区别？

$1 < n < 21, 1 \leq r \leq n$

排列问题

通过仔细观察，我们会发现该题和前面的题最大的区别是，前面的题当前只需要满足该数没有被用过就可以了。而该题不是。

1	2	3
1	2	4
1	2	5
1	3	4
1	3	5
1	4	5
2	3	4
2	3	5
2	4	5
3	4	5

对于任意选出的固定的三个数而言，只算一种可能。而前面问题的组合问题，只要有一个数的位置不相同，那么就算一种方案。

比如选出了1 2 3三个数，只算一种可能，其他组合，比如3 2 1，1 3 2这些都算重复的。如果只是按照前面的做法就会重复计算

排列问题

对于这一类问题，如果位置不同也算一种方案，为了避免重复，我们最好让**下一次选的数不小于当前选的数**，这样最终我们选出来的序列就是一个不下降序列，这样就避免了重复，也不会有遗漏。

所以我们每个位置选数的条件就是：

- (1) 选的数在要求的范围之内
- (2) 该数以前没有被选过
- (3) 该数不比前一个选的数小(本题应该是大于)

我们可以想到两种方法来处理问题：

- (1) 存储当前数，下一次判断是否小于上一次的数
- (2) 把当前选的数作为另一个参数传递给下一个函数

排列问题

```
void dfs(int step, int k)
{
```

用k记录上一次选的数

```
    if (step == r + 1)
    {
```

输出结果并且返回

```
    }
```

```
    for (int i = k + 1; i <= n; i++)
    {
```

该次选数的范围是 $i > k$

```
        if (book[i] == 0)
        {
```

```
            book[i] = 1;
```

```
            dfs(step + 1, i);
```

```
            book[i] = 0;
```

把该次选的数传递下去

```
        }
```

```
    }
```

```
    return ;
```

```
}
```

`dfs(1, 0);` // 当前位置，可选数的最小值

排列问题

```
void dfs(int step)
{
```

```
    if (step == r + 1)
    {
```

输出结果并且返回

```
    }
```

```
    for (int i = a[step - 1] + 1; i <= n; i++)
    {
```

```
        if (book[i] == 0)
        {
```

下一个数必须大于上一个数

```
            book[i] = 1;
            a[step] = i;
            dfs(step + 1);
            book[i] = 0;
```

用a[step]存储当前选的数

```
        }
```

```
    }
    return ;
```

最好用第一种办法

```
} dfs(1); // 当前位置，可选数的最小值
```

放苹果

把M个同样的苹果放在N个同样的盘子里，允许有的盘子空着不放，问共有多少种不同的分发（5, 1, 1和1, 1, 5是同一种方法）

输入：

7 3

输出：

8

M, N ≤ 10

很明显，该题和前面的排列问题是一样的，对于同样的组合不同的位置只算一种情况。所以需要考虑上一次盘子放的苹果是多少。非递减这样放下去才不会重复

放苹果

这道题可以递推来做，但是递推公式很难想。

```
for(int i=1;i<=n;i++)//盘子数量
```

```
{  
    for(int j=1;j<=m;j++)//苹果数量
```

```
{
```

```
    if(i>j)
```

```
    f[i][j]=f[j][j];
```

```
    else
```

```
    f[i][j]=f[i-1][j]+f[i][j-i];
```

```
}
```

如果盘子数量大于苹果数量，说明肯定有空盘子，有没有空盘子对方案数没影响

如果苹果数大于盘子数，还是可以分为所有盘子都放了苹果和至少一个盘子没有放苹果这两种情况

如果所有盘子都放了苹果，那么每个盘子减少一个苹果不会影响方案数。如果至少有一个盘子没放苹果，少一个盘子也不会有影响

能想到的都是
dalao

放苹果

遇到这种题的时候，我们会发现还是递归解题要方便点，很多题，都可以用递推和递归来做，并且递推解题效率要远高于递归，但是关键是想不出递推公式也是白搭

我们再想一下，该题和前面我们做过的题有什么区别。

我们会发现，每次放置了苹果之后，不但下一次可以放苹果的范围的起点变了，终点也改变了。起点变为 $\geq$ 上一次放的数量， $\leq$ 剩下苹果的数量。

放苹果

```
void dfs(int step,int k,int sheng)
{
    if(step==n+1)
    {
        if(sheng==0)
            输出
    }
    for(int i=k;i<=sheng;i++)
    {
        a[step]=i;
        dfs(step+1,i,sheng-i);
        a[step]=0;
    }
    return ;
}
```

当 $step==n+1$ 的时候，并不是这种方案就合理了，而是苹果放完了才合理，不然就不是合理的方案，直接返回

每次传递剩余的苹果数量，由于是传递，他本来的数量并不会改变，所以不影响回溯

最好不要直接写成 $m=m-i$ 这种形式，如果这样写，回溯的时候必须要加上 $m=m+i$ ，不然就错误了

数的计算

我们要求找出具有下列性质数的个数(包含输入的自然数 n):
先输入一个自然数 n ($n \leq 1000$), 然后对此自然数按照如下方法进行处理:

1. 不作任何处理;
2. 在它的左边加上一个自然数, 但该自然数不能超过原数的一半;
3. 加上数后, 继续按此规则进行处理, 直到不能再加自然数为止.

输入:

6

输出:

6

(6, 36, 26, 16, 136, 126)

$n \leq 1000$

数的计算

第一种方法：递推

如果 i 是奇数

$f[i]=f[i-1];$

//当 i 是奇数时， i 和 $i-1$ 的一半是相同的

如果 i 是偶数

$f[i]=f[i-1]+f[i/2];$

//当 i 是偶数时， $f[i]$ 与 $f[i-1]$ 的一半不相同。
并且 $f[i-1]$ 没有 $i/2$ 这个因子，所以我们再加上所有的以 $i/2$ 开头的方案数。

数的计算

第二种方法：回溯

```
void dfs(int m)
{
    if(m==1)
        return ;
    for(int i=m/2;i>=1;i--)
    {
        N++;
        dfs(i);
    }
    return ;
}
```

dfs(n);

注意没有包含n本身这种情况，所以初始化N=1;

注意是每分一次，方案数增加一次，比如6，第一次分成36，继续分解为136. 都是两种方案。

注意不是m==1时，才N++;

数的计算

但是由于该题 $n \leq 1000$, 方案数会有很多, 所以会导致爆栈。我们需要做一个优化

当 $n=10$ 的时候:

第一次可以分为5, 4, 3, 2, 1。

如果第一次分成5,

那么第二次可以分成2, 1。

如果第一次分成4,

拿了么第二次可以分成2, 1。

如果第一次分成3,

第二次可以分成1。

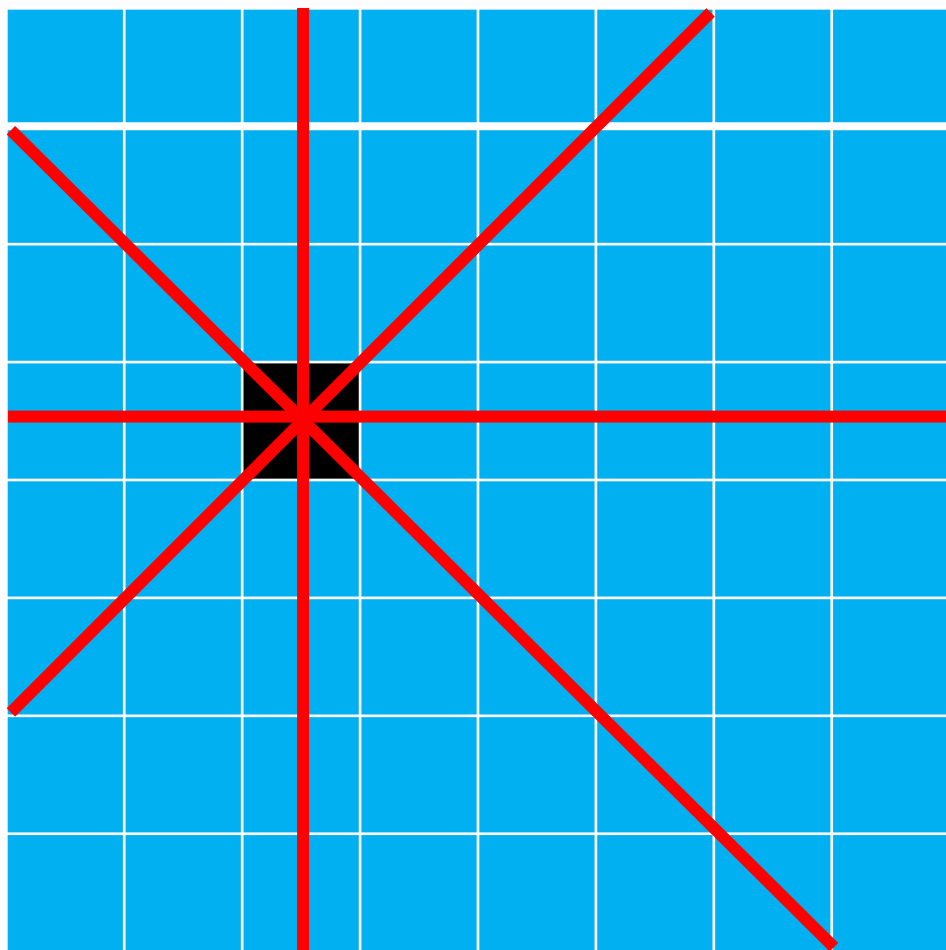
如果第一次分成2,

第二次可以分成1。

我们会发现, 很多时候都是重复计算的, 特别是比较小的, 可能会重复计算成千上万次,

记忆化搜索

八皇后问题



在一个 8×8 的格子里面放入8个国际象棋里面的皇后，要求每个皇后之间不可以互相攻击(皇后的攻击范围是该行，该列和所对应的两条对角线)

比如黑色格子是皇后的位置，那么他的攻击范围是红色线条的区域

八皇后问题

很明显，如果要摆放八个皇后并且满足要求，至少需要满足的条件是每一行一个，每一列一个。最后再判断是否属于同一个对角线。对于放入的每一个皇后。我们都需要判断：

- (1) 该行是否有其他皇后
- (2) 该列是否有其他皇后
- (3) 该位置对应的对角线是否有其他皇后。

为了让问题更加简单，我们直接放一个之后换一行放下一个皇后，这样就不需要再判断是否在同一行了。

同样的，每放一个皇后我们标记该位置的列号，放下一个皇后的时候判断该列是否被标记过。

八皇后问题

```
void dfs(int hang)
```

```
{
```

```
    if(hang==9)
```

```
    {
```

```
        N++; return ;
```

```
    }
```

```
    for(int i=1;i<=8;i++)
```

```
    {
```

```
        if(book[i]==0&&对角线没有皇后)
```

```
        {
```

```
            book[i]=1;
```

```
            标记两条对角线;
```

```
            dfs(hang+1);
```

```
            book[i]=0;
```

```
            取消对角线标记;
```

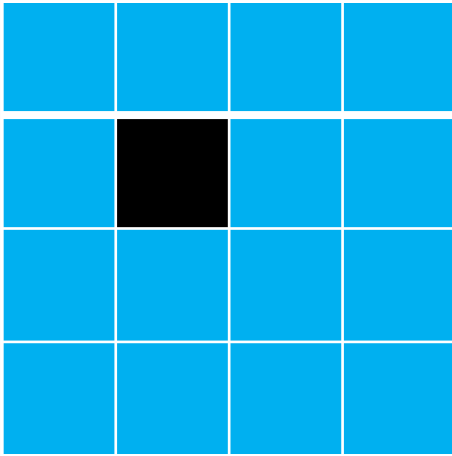
```
        }
```

```
    }
```

```
}
```

关键是怎么判断对角线有没有其他皇后，怎么标记对角线

八皇后问题



对于一个任意一个 $n \times n$ 的方格，当前放皇后的位置是 (i, j) 。

那么左对角线的方格坐标分别是 $(i-k, j+k)$ 或者 $(i+k, j-k)$ 。

右对角线的方格坐标分别是 $(i-k, j-k)$ 或者 $(i+k, j+k)$

所以我们该如何去标记某个位置对应的对角线？？？

对于以 (i, j) 为中心的左对角线上的点 (x, y)

$$x+y=i+j$$

对于以 (i, j) 为中心的右对角线上的点 (x, y)

$x-y=i-j$ 但是考虑到数组下标 ≥ 0 ，我们可以加上 n

工作分配

n 个人从事 n 项工作，每人只能从事一项，求最佳安排使效益最高。

设有A, B, C, D, E人从事J1, J2, J3, J4, J5五项工作，每人只能从事一项，他们的效益如下

当 A从事J5, B从事J3, C从事J4, D从事J1, E从事J2时收益最大值50

输入 n 和矩阵

输出最大效益和方案

输入

5

13 11 10 4 7

13 10 10 8 5

5 9 7 7 4

15 12 10 11 5

10 11 8 8 4

输出

50

7

10

7

15

11

	J1	J2	J3	J4	J5
A	13	11	10	4	7
B	13	10	10	8	5
C	5	9	7	7	4
D	15	12	10	11	5
F	10	11	8	8	4

工作分配

和前面的回溯方法一样。对于每一个人而言。只可以选前面没有选过的工作。

但是有差别的地方是，方案数有很多种，而我们需要选出最优的（和最大）的那种。即选出一种合理的方案后，比较当前方案的效益和当前方案之前的最大效益，如果大于，则更新最大效益再返回，如果小于等于则直接返回

```
if(sum>maxn)  
    maxn=sum;  
return;
```

最短时间

有n个人需要一起共同完成n项工作。每个人都可以完成其中任意一项工作，但是每个人完成每项工作都需要一个不同的时间。现给出n个人完成n项工作分别用的时间，问：n个人完成n项工作需要的最小时间，每个人只允许完成其中一项工作。

输入

5

1 3 4 6 2

3 6 3 5 8

7 4 8 2 4

9 5 3 7 8

4 3 6 4 8

输出：

13

(5 1 4 3 2)

$n \leq 15$

最短时间

该题和前面的题一样。都是选出一种合理的方案数之后还需要判断该方案数需要的时间是否小于前面的最短时间，如果是，则更新最小时间。但是他们仔细思考一下，有没有什么办法可以让代码更加优化呢？

假设当前的最小值时 $minn$ ，当我找下一种方案的时候，当我递归到第 k 个人的时候，此时该方案下前 k 个人需要的时间之和就已经 $\geq minn$ ，那我们就在这个时候可以直接把这种方案PASS了，不需要再继续往后面看了，因为每个工人用的时间不可能为负数。加上后面工人需要的总时间肯定比当前的总时间大。当前总时间都不是最优的，最终的总时间肯定也不是最优的。

剪枝